第30课 递归思想（二）——汉诺塔、阶乘

一、课程信息

项目	内容
课次	第 30 课 / 共 40 课
主题	递归思想（二）——阶乘、斐波那契数列、汉诺塔
前置知识	第29课：递归基本概念（递归三要素、简单递归函数）
后续衔接	第31课：栈与队列（STL）

教学目标（可检验）

1. **能独立写出**阶乘和斐波那契数列的递归函数，并手动推演 `factorial(5)` 的完整调用过程。
2. **能用自己的话说出**汉诺塔递归三步策略（搬上面、移最大、搬回来），并手动演示 3 个盘的全部 7 步操作。
3. **能独立编写**汉诺塔递归程序，输入 n 输出搬运步骤，并说出至少一个"递归 vs 循环"的区别。

二、教学重点与难点

重点

- 阶乘递归的实现与调用过程理解
- 汉诺塔问题的递归拆解思路（三步法）
- 汉诺塔递归代码的编写

难点

- 汉诺塔"先搬 n-1 个盘"的递归信任——不需要想清楚每一步，只要相信递归能帮你搞定
- 理解递归调用时参数（起点、中转、终点）的角色互换
- 从"手动模拟"到"写出代码"的跨越

三、课堂时间分配（共 100 分钟）

时间段	时长	内容	形式
0′–5′	5 min	复习热身：回忆递归三要素，快速口答	师生互动
5′–20′	15 min	讲解一：阶乘递归 + 斐波那契递归	讲授 + 板书推演
20′–30′	10 min	练习一：阶乘 & 斐波那契上机编码	学生实操
30′–55′	25 min	讲解二：汉诺塔问题（故事引入实物演示递归思路代码）	讲授 + 实物演示
55′–75′	20 min	练习二：汉诺塔上机编码 + 变式练习	学生实操
75′–85′	10 min	讲解三：递归 vs 循环对比	讲授 + 讨论
85′–95′	10 min	综合练习 + 答疑	学生实操
95′–100′	5 min	总结 + 作业布置 + 下节课预告	师生互动

> 讲解段最长为 25 分钟（汉诺塔部分），其余讲解段均不超过 15 分钟。

四、讲稿提纲

第一段（5′–20′）：阶乘递归 & 斐波那契递归

4.1 复习开场（0′–5′）

> 同学们，上节课我们学了一个神奇的东西——递归！谁还记得递归的三要素？
> 对！**函数自己调用自己、每次问题变小、有一个终止条件（出口）**。
> 今天我们要用递归来解决三个经典问题，难度一个比一个大，像闯关一样！

4.2 阶乘递归（5′–12′）

> **故事引入：排队领奖品**
>
> 想象一下，学校发奖品，5 个同学要排成一队。排队方式有多少种呢？
>
> - 第一个位置有 5 个人可以选
> - 第二个位置有 4 个人可以选
> - 第三个位置有 3 个人
> - ……
>
> 所以答案是 5 × 4 × 3 × 2 × 1 = 120 种。这就是 **5 的阶乘**，写成 `5!`。
>
> 现在我们换一种想法：
>
> - 想知道 5 个人怎么排？ 先让我知道 4 个人怎么排，然后乘以 5
> - 想知道 4 个人怎么排？ 先让我知道 3 个人怎么排，然后乘以 4
> - ……一直到 0 个人怎么排？ 只有 1 种方式（谁都不排，就 1 种）
>
> 看！这就是递归啊！大问题拆成小问题，最后有个出口！
>
> **公式：** `n! = n × (n-1)!`，出口：`0! = 1`

板书推演 factorial(4)：

```
factorial(4)
= 4 * factorial(3)
= 4 * 3 * factorial(2)
= 4 * 3 * 2 * factorial(1)
= 4 * 3 * 2 * 1 * factorial(0)
= 4 * 3 * 2 * 1 * 1
= 24
```

4.3 斐波那契数列（12′–20′）

> **故事引入：兔子生兔子**
>
> 很久很久以前，有一个数学家叫斐波那契，他研究兔子繁殖：
>
> - 第 1 个月：1 对小兔子
> - 第 2 个月：还是 1 对（小兔子还没长大）
> - 第 3 个月：变成 2 对（小兔子长大生了 1 对）
> - 第 4 个月：变成 3 对
> - 第 5 个月：变成 5 对
>
> 规律是什么？ **1, 1, 2, 3, 5, 8, 13, 21, …**
>
> 每个数 = 前两个数加起来！用递归来说就是：
>
> - `fib(n) = fib(n-1) + fib(n-2)`
> - 出口：`fib(1) = 1, fib(2) = 1`
>
> 小提示：斐波那契递归算大数字会很慢很慢，之后第38课我们学"动态规划"会有更快的方法！今天先学会递归写法就好。

第二段（30′–55′）：汉诺塔问题

4.4 故事引入

> **古老的传说**
>
> 在印度有一座寺庙，庙里有三根金柱子（我们叫它 A、B、C），A 柱子上从小到大叠着 64 个金盘子，最小的在最上面，最大的在最下面。
>
> 和尚们的任务是：把所有盘子从 A 柱子移到 C 柱子上，但有两个规则：
>
> 1. **一次只能移一个盘子**
> 2. **大盘子绝不能放在小盘子上面**
>
> 传说如果64个盘子全部移完，世界就会毁灭……（别担心，那需要 18446744073709551615 步，就算一秒移一个也要五千多亿年呢！）
>
> 今天我们不搞 64 个，先从 1 个、2 个、3 个开始！

4.5 实物演示（准备 3 本不同大小的书 / 3 个不同大小的杯子）

> **请一位同学上台来试试！**
>
> 先试 **1 个盘**（1 本书）：直接从 A 搬到 C。1 步搞定！
>
> 再试 **2 个盘**：
>
> 1. 小盘 AB
> 2. 大盘 AC
> 3. 小盘 BC
>
> 一共 3 步！
>
> 再试 **3 个盘**（全班一起喊步骤）：
>
> 1. 小盘 AC
> 2. 中盘 AB
> 3. 小盘 CB
> 4. 大盘 AC 最大的盘到位了！
> 5. 小盘 BA
> 6. 中盘 BC
> 7. 小盘 AC
>
> 一共 7 步！谁发现规律了？ 1步、3步、7步…… **每次都是上一次的两倍再加1！**

4.6 递归思路讲解

> 3 个盘看起来很复杂对不对？但如果我们用递归的思想，就变得超级简单——只需要 **三步**！
>
> 想象你是一个搬家公司的老板，要把 n 个盘子从 A 搬到 C：
>
> **第一步：** 先请你的"副手"（递归）把上面 n-1 个小盘子从 A 搬到 B（C 当中转站）
>
> **第二步：** 你亲自把最底下的大盘子从 A 搬到 C（这是你唯一要干的活！）
>
> **第三步：** 再请你的"副手"（递归）把那 n-1 个小盘子从 B 搬到 C（A 当中转站）
>
> **"等等，副手怎么知道怎么搬 n-1 个盘子？"**
>
> 副手也用同样的方法！他也请自己的副手搬 n-2 个……一直到只剩 1 个盘子的时候，直接搬就行了！这就是递归的出口！
>
> **记住这个口诀：搬上面 移最大 搬回来**

4.7 代码讲解

> 注意看代码里的参数：`from`（从哪来）、`aux`（中转站）、`to`（到哪去）
>
> 每次递归调用的时候，这三个角色会互相交换位置——这是汉诺塔代码最巧妙的地方！

第三段（75′–85′）：递归 vs 循环

> 有同学会问：用 for 循环也能算阶乘啊，为什么要用递归？
>
> 好问题！我们来对比一下：
>
> | | 递归 | 循环 |
> |---|---|---|
> | 像什么 | 套娃，自己装自己 | 跑圈，一圈又一圈 |
> | 阶乘 | 好写 | 也好写 |
> | 汉诺塔 | 非常简洁 | 循环写起来很痛苦 |
> | 速度 | 有时候慢（比如斐波那契） | 通常快一些 |
> | 什么时候用递归？ | 问题能自然地拆成"同样结构的小问题"时 | 简单重复的时候 |
>
> **总结：能用循环就循环，但有些问题（比如汉诺塔），递归是最自然、最优雅的解法！**

七、常见错误与应对

错误 1：忘写递归出口 无限递归 程序崩溃

```
// 错误写法：没有出口！
long long factorial(int n) {
 return (long long)n * factorial(n - 1); // n 不断减小但永远不会停
}
```

现象： 程序运行后卡住或直接崩溃，报 `Segmentation fault` 或 `Stack overflow`。

应对：

> "递归就像下楼梯，出口就是一楼。如果没有一楼，你就会一直往地下走，掉进地心啦！写递归的第一件事——**先写出口**！"

错误 2：汉诺塔参数顺序写错

```
// 错误：第一步应该把 n-1 个盘搬到 aux，但 to 和 aux 写反了
hanoi(n - 1, from, aux, to); // 这样搬到了 to 而不是 aux！
```

应对：

> "记住口诀：第一步——起点不变，终点和中转互换；第三步——终点不变，起点和中转互换。写完后拿 n=2 手动验一遍！"

错误 3：阶乘用 int 导致溢出

```
// 错误：int 最大约 21 亿，13! = 6227020800 就溢出了
int factorial(int n) {
 if (n == 0) return 1;
 return n * factorial(n - 1);
}
```

应对：

> "阶乘增长特别快，像滚雪球一样越滚越大。20! 已经是一个 19 位数了！所以一定要用 `long long`。"

错误 4：斐波那契出口只写了一个

```
// 错误：只写了 n==1 的出口
int fib(int n) {
 if (n == 1) return 1;
 return fib(n - 1) + fib(n - 2); // 当 n=2 时，fib(0) 没有出口!
}
```

应对：

> "斐波那契数列需要**两个**出口，因为它每次要回头看**两个**小朋友。一个出口不够用！`n==1` 和 `n==2` 都要写。"

错误 5：对递归"不信任"，试图追踪每一层调用

现象： 学生在写汉诺塔时，总想把每一步都想清楚，越想越乱，最后放弃。

应对：

> "递归就像请快递小哥帮你送包裹——你只要告诉他'从这儿送到那儿'，不用管他走哪条路。你要**信任递归**！只要出口对、递归公式对，递归一定能帮你搞定！"

九、下节课预告

> 同学们，今天我们用递归解决了阶乘、斐波那契数列和汉诺塔三个经典问题。递归是一个非常强大的武器，以后学搜索（DFS/BFS）和动态规划的时候还会经常用到它！
>
> 下节课（第31课），我们要学习两个新的数据结构——**栈和队列**。
>
> - **栈**就像一摞盘子：最后放上去的最先拿走（后进先出）
> - **队列**就像食堂排队：先来的先打饭（先进先出）
>
> 有意思的是，递归的背后其实就藏着一个"栈"！下节课你们就会明白，为什么递归忘写出口会"栈溢出"了。
>
> 回去把今天的作业做完，下节课见！